

— ARQUITECTURA DE PAGOS · RESILIENCIA

Un pago que **nunca** se pierde.

Ni cuando el procesador externo no responde.

● PENDIENTE

● EN_PROCESO

● INCIERTO

● PAGADO

— EL PROBLEMA

Lo pienso en tres momentos.

Recepción Nunca perder el pago

Persisto con una **idempotency-key** única antes de hablar con nadie. Misma key → mismo resultado.

Envío No colgar al usuario

Respondo “recibido” y cobro **asíncrono**. Timeout + breaker; si falla, ruteo al alternativo.

Incertidumbre El caso feo: timeout

Un timeout no es un fallo: es “no sé”. Queda **INCIERTO** y disparo conciliación.

“Un timeout no significa que falló.

Significa que no sé.

Y nunca asumo.”

– la regla que sostiene todo el diseño

— LOS PILARES

Cuatro decisiones.



Idempotency-key end-to-end

La misma key del cliente a la API y de la API al procesador.



Timeout $\neq$ fallo $\rightarrow$ INCIERTO

Ante timeout no se asume nada. Queda en verificación.



Conciliación = fuente de verdad

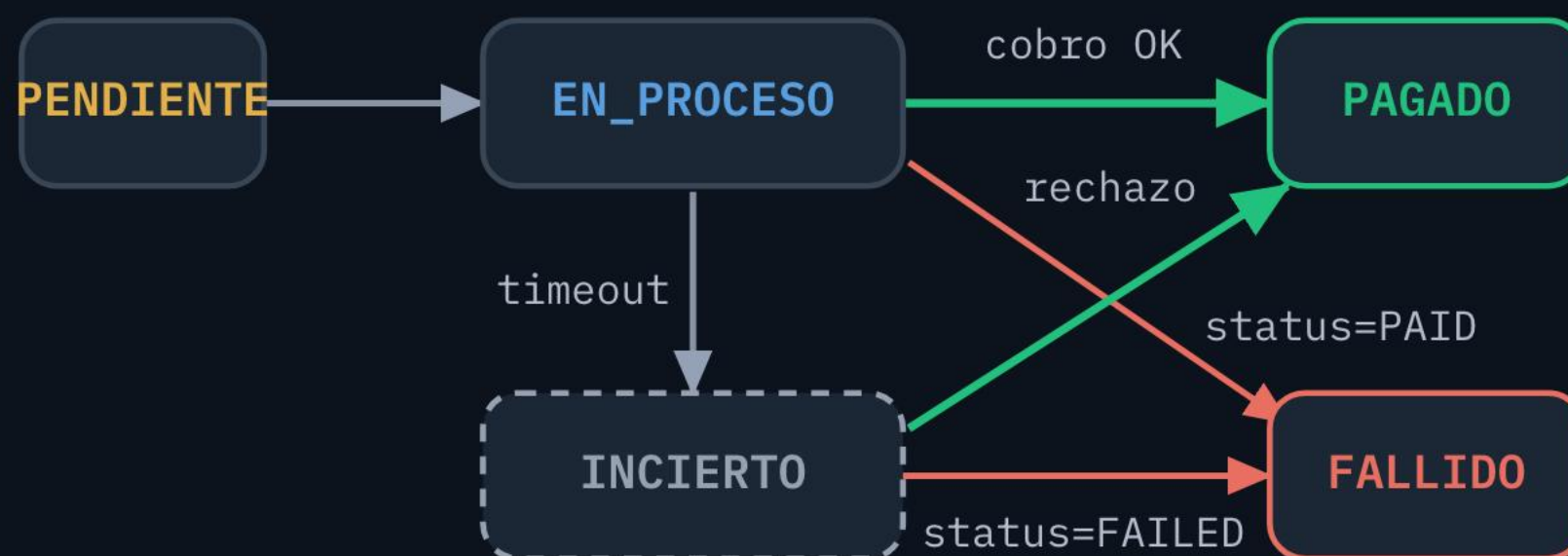
El estado del dinero lo define el procesador, no mi código.



Circuit breaker + ruteo

Si el procesador falla, abro el breaker y ruteo al alternativo.

El dinero nunca salta a **PAGADO** sin el procesador.



Una máquina de estados valida cada transición. Hasta que la conciliación no confirma, la plata no está cobrada.

Dos patrones bonus.

📦 OUTBOX · no perder eventos

El pago y su evento se persisten en la **misma transacción de BD**. Si el proceso se cae, un worker lo retoma. Nada se pierde.

🔄 SAGA · ciclo de vida explícito

Máquina de estados con transiciones validadas y **compensaciones**. Los estados terminales no vuelven atrás.

— EL PROYECTO

Diseño, arquitectura y código real.

.NET 10

EF Core

Polly v8

Outbox

Circuit Breaker

Saga

24 tests ✓

¿Vos qué hacés cuando el procesador te da timeout?

→ github.com/jbaldi/apiTransacciones